

```

// Acts as a shorthand list of error functions
// Restructuring the ordering at this point may change an error's ID,
// However, the code is set up such that it can support such a change
var (
    /**
     * Parsing related error functions.
     */
    // Argument parsing errors
    OPTIONS_ARGUMENT_START_MISSING = CustomErrorWrapper("The
opening square bracket '[' is missing from the argument.", "You should have an
associated opening square bracket '[' for your closing square bracket ']'".
\n\nExamples:\n- \\documentclassdraft] could have this specific error fixed by
adding '[' between 'class' and 'draft', creating \\documentclass[draft] -- the
final result may be \\documentclass[draft]{article}\n- \
\\includegraphicsartifact] could have this specific error fixed by adding ']'
between 'graphics' and 'artifact', creating \\includegraphics[artifact] -- the
final result may be \\includegraphics[artifact]{image.png}", newParseError)
    OPTIONS_ARGUMENT_END_MISSING = CustomErrorWrapper("The
closing square bracket ']' is missing from the argument.", "You should have an
associated closing square bracket ']' for your opening square bracket '['.
\n\nExamples:\n- \\documentclass[draft could have this specific error fixed by
adding ']' to the end, creating \\documentclass[draft] -- the final result may
be \\documentclass[draft]{article}\n- \\includegraphics[alt={Some alt text}
could have this specific error fixed by adding ']' to the end, creating \
\\includegraphics[alt={Some alt text}] -- the final result may be \
\\includegraphics[alt={Some alt text}]{image.png}", newParseError)
    CLASS_ARGUMENT_START_MISSING = CustomErrorWrapper("The
opening curly braces '{' is missing from the argument.", "You should have an
associated opening curly brace '{' for your closing curly brace '}'".
\n\nExamples:\n- \\documentclassarticle} could have this specific error fixed by
adding '{' between 'class' and 'article', creating \\documentclass{article}\n- \
\\includegraphicsimage.png} could have this specific error fixed by adding '{'
between 'graphics' and 'image.png', creating \\includegraphics{image.png}",
newParseError)
    CLASS_ARGUMENT_END_MISSING = CustomErrorWrapper("The
closing curly braces '}' is missing from the argument.", "You should have an
associated closing curly brace '}' for your opening curly brace '{'.
\n\nExamples:\n- \\documentclass{article could have this specific error fixed by
adding '}' to the end, creating \\documentclass{article}\n- \
\\includegraphics{image.png could have this specific error fixed by adding '}' to
the end, creating \\includegraphics{image.png}", newParseError)
    INNER_ARGUMENT_END_MISSING = CustomErrorWrapper("A closing
argument, '}' or ']' is missing from the argument.", "You should have an
associated ending curly brace '}' or square bracket ']' in your inner argument.
The inner argument is the content within your arguments.\n\nExamples:\n- \
\\includegraphics[alt={some alt text}{image.png} could be fixed by adding '}'
directly after 'argument', creating \\includegraphics[alt={some alt text}]
{image.png}\n- \\DocumentMetadata{tagging=on,tagging-setup={activate/all} could
be fixed by adding '}' to the end, creating \
\\DocumentMetadata{tagging=on,tagging-setup={activate/all}}", newParseError)
    INVALID_ARGUMENT_CONTENT_KEY_VALUE_FORMAT = CustomErrorWrapper("This
argument is not in key-value pair form. A pair consists of a key and value
separated by '=', with each pair separated by ',', "The argument should be in
key-value form.\n\nA pair consists of 2 values separated by '=', where the right
value could be surrounded by curly braces '{' with content within it.\nAn
argument in key-value form contains many pairs, each separated by commas ','.
\n\nExamples:\n- \\DocumentMetadata{food} is not in key-value form, whereas \
\\DocumentMetadata{tagging=on} is.\n- \\includegraphics[good]{image.png} does not
have its optional parameter, surrounded by square brackets '[]', in key-value
form. However, \\includegraphics[alt={Some alt text}]{image.png} does have its
optional parameter in key-value form.", newParseError)
    KEY_EMPTY = CustomErrorWrapper("The key
provided is empty.", "An empty key has been provided to the argument. This can
be fixed by simply adding a key to the relevant pair.\nExamples:\n- \

```

```

\DocumentMetadata{=on} is missing the key 'tagging', where the fixed version is
\\DocumentMetadata{tagging=on}\n- \\includegraphics[={Some alt text}]{image.png}
is missing the key 'alt', where \\includegraphics[alt={Some alt text}]{
image.png} is the fixed version", newParseError)
    VALUE_EMPTY = CustomErrorWrapper("The value
provided is empty.", "An empty value has been provided to the argument. This can
be fixed by simply adding a value to the relevant pair.\nExamples:\n- \
\DocumentMetadata{tagging=} is missing the value 'on' or 'off', where the fixed
version can be \\DocumentMetadata{tagging=on}\n- \\includegraphics[alt=]
{image.png} is missing the value for the key 'alt', where \
\includegraphics[alt={Some alt text}]{image.png} is a valid fixed version",
newParseError)
    INNER_ARGUMENT_MISMATCH = CustomErrorWrapper("There was
a mismatch from the starting and ending braces.", "There was a mismatch from the
starting and ending braces for the inner argument. The inner argument is the
content within an argument. This means two incompatible grouping methods have
been using, such as an opening square bracket '[' together with a closing curly
brace '}'.\n\nExamples:\n- \\includegraphics[alt={Some alt text}]{image.png}
could be fixed by using \\includegraphics[alt={Some alt text}]{image.png}\n- \
\includegraphics[alt=[Some alt text]]{image.png} could be fixed by using \
\includegraphics[alt={Some alt text}]{image.png}", newParseError)

    // Group parsing errors
    GROUP_NAME_MISMATCH = CustomErrorWrapper("The class argument within \
\begin line did not match the class argument within \\end line.", "The ending
group name did not match the starting group name. This can be fixed by either
using the starting group name within the ending group, or vice versa.\nExamples:
\n- \\begin{tabular}...\end{itemize} can be fixed by replacing 'itemize' with
'tabular', creating \\begin{tabular}...end{tabular}.\n- \\begin{tabular}...\
\end{itemize} can be fixed by replacing 'tabular' with 'itemize', creating \
\begin{itemize}...end{itemize}.", newParseError)
    GROUP_BEGIN_MISSING = CustomErrorWrapper("The matching \\begin line could
not be found for the \\end line.", "There is no beginning for the group. This
can be fixed by adding a beginning for the group.\n\nExample:\n- ... \
\end{tabular} by itself by can be fixed by adding a \\begin{tabular} to the
front, creating \\begin{tabular}...\end{tabular}", newParseError)
    GROUP_END_MISSING = CustomErrorWrapper("The matching \\end line could
not be found for the \\begin line.", "There is no ending for the group. This can
be fixed by adding an ending for the group.\n\nExample:\n- \\begin{tabular}...
by itself can be fixed by adding a \\end{tabular} to the end, creating \
\begin{tabular}...\end{tabular}", newParseError)

    // Server-responsible parsing errors
    SERVER_RESPONSIBLE_STACK_EMPTY =
CustomErrorWrapper("Something went wrong on the server's end. The grouping stack
is empty. Please report this to the server owner.", "Something went wrong on the
server's end. The grouping stack is empty. Please report this to the server
owner.", newParseError)
    SERVER_RESPONSIBLE_STACK_CONTAINS_NON_GROUP_ELEMENT =
CustomErrorWrapper("Something went wrong on the server's end. The grouping stack
contained a non-group element. Please report this to the server owner.",
"Something went wrong on the server's end. The grouping stack contained a non-
group element. Please report this to the server owner.", newParseError)
    SERVER_RESPONSIBLE_NON_COMPONENT_DATA_STRUCTURE =
CustomErrorWrapper("Something went wrong on the server's end. The data structure
for a LaTeX line did not return one that is supported. Please report this to the
server owner.", "Something went wrong on the server's end. The data structure
for a LaTeX line did not return one that is supported. Please report this to the
server owner.", newParseError)
    SERVER_RESPONSIBLE_TABLE_NOT_FOUND =
CustomErrorWrapper("Something went wrong on the server's end. The tabular group
could not be found. Please report this to the server owner.", "Something went
wrong on the server's end. The tabular group could not be found. Please report
this to the server owner.", newParseError)

```

```

SERVER_RESPONSIBLE_INVALID_DOCUMENT_CLASS =
CustomErrorWrapper("Something went wrong on the server's end. A non-document
class argument was provided as one. Please report this to the server owner.",
"Something went wrong on the server's end. A non-document class argument was
provided as one. Please report this to the server owner.", newParseError)

// conversion parsing errors
CANNOT_CONVERT_TO_PDF = CustomErrorWrapper("The file was not converted
into .pdf format. Please compile it on a TeX engine to identify the cause.", "An
error that is not handled on LinTeX's end had occurred during the compilation of
the document. You should use a dedicated environment to debug this error, such
as TeX Live or Overleaf.", newParseError)

/**
 * Structure related error functions
 */
// Missing essential element structure errors
DOCUMENT_CLASS_NOT_FOUND = CustomErrorWrapper("Missing \
\documentclass line.", "There was no \documentclass line detected in the
document. This is necessary for the document to compile.\n\nExamples:\n- \
\documentclass[draft]{article} is a valid example\n- \documentclass{book} is a
valid example", newStructureError)
SEVERAL_DOCUMENT_CLASSES_FOUND = CustomErrorWrapper("You cannot
have more than 1 document class within a single document.", "There should only
ever be 1 \documentclass command within a single document. This error can be
alleviated by removing all document class commands that are not essential to the
document.\n\nExample:\n- \documentclass{article}\documentclass{book} could be
fixed by removing the 'book' document class, leaving only \
\documentclass{article}", newStructureError)
DOCUMENT_CLASS_REQUIRED_ARGUMENT_MISSING = CustomErrorWrapper("\
\documentclass should have at least 1 required argument. For example, '\
\documentclass{article}' is a valid example.", "The document class currently
contains no required arguments indicating its type. This can be alleviated by
providing it with a valid typing.\n\nExample:\n- \documentclass could be fixed
by adding the 'article' or 'book' types, creating \documentclass{article} and \
\documentclass{book} respectively.", newStructureError)
DOCUMENT_CONTENT_GROUP_NOT_FOUND = CustomErrorWrapper("Missing \
\begin{document} and \end{document} lines.", "The body of the document should
be contained within a 'document' group. Content outwith this group is considered
to be within the preamble.\n\nExample:\n- \includegraphics[alt={Some alt text}]
{image.png} cannot exist outside the 'document' group, so \begin{document} ...
\includegraphics[alt={Some alt text}]{image.png} ... \end{document} is a valid
solution if it exists outside the relevant group.", newStructureError)
DOCUMENT_CONTENT_GROUP_NOT_DOCUMENT_TYPE = CustomErrorWrapper("Incorrect
group name used for the document content.", "It was expected to receive a
'document' group indicating the body of the document, but another grouping type
was provided instead. This could be caused by forgetting to include the
'document' group or having some typo exist somewhere.\n\nExample:\n- \
\begin{tabular} ... \end{tabular} could require being encapsulated in a
'document' group, creating \begin{document} ... \begin{tabular} ... \
\end{tabular} ... \end{document}", newStructureError)

// Missing required arguments structure errors
MISSING_BEGIN_GROUP_NAME = CustomErrorWrapper("You must have at least one
required argument '{ }' with \begin lines.", "A group beginning has no name
provided to it. This could be caused by a lack of required arguments, surrounded
by '{ }'. \n\nExample:\n- \begin by itself is invalid, whereas \begin{tabular}
is valid as it was assigned the 'tabular' group name.", newStructureError)
MISSING_END_GROUP_NAME = CustomErrorWrapper("You must have at least one
required argument '{ }' with \end lines.", "A group ending has no name provided
to it. This could be caused by a lack of required arguments, surrounded by '{ }'.
\n\nExample:\n- \end by itself is invalid, whereas \end{tabular} is valid as
it was assigned the 'tabular' group name.", newStructureError)

```

```

// Containing group constructs within non-grouping contexts
PREAMBLE_CONTAINS_GROUP = CustomErrorWrapper("The preamble cannot
contain any grouping constructs such as \\begin or \\end.", "The preamble of the
document, the document outwith the 'document' group, cannot contain grouping
constructs in its outermost layer. This was likely caused by misplacing a
grouping construct that should be within the 'document' group.\nExample:\n- \
\\usepackage{...} ... \\begin{tabular} ... \\end{tabular} ... \
\\begin{document} ... \\end{document} could be fixed by moving the 'tabular'
group within the 'document' group, creating \\usepackage{...} ... \
\\begin{document} ... \\begin{tabular} ... \\end{tabular} ... \\end{document}",
newStructureError)

PREREQUISITE_CONTAINS_GROUP = CustomErrorWrapper("The content before the
document class cannot contain any grouping constructs such as \\begin or \
\\end.", "The prerequisite content of the document, the document before the
documentclass, cannot contain grouping constructs. This was likely caused by
misplacing a grouping construct that should be within the 'document' group.
\nExample:\n- \\begin{tabular} ... \\end{tabular} ... \\documentclass{...} ... \
\\begin{document} ... \\end{document} could be fixed by moving the 'tabular'
group within the 'document' group, creating ... \\documentclass{...} ... \
\\begin{document} ... \\begin{tabular} ... \\end{tabular} ... \\end{document}",
newStructureError)

// Extra latex code where it shouldn't belong
DOCUMENT_CONTENT_ALREADY_EXISTS = CustomErrorWrapper("You cannot have more
than one document content component.", "You can only have one 'document' group
within the document. This error was caused by 2 or more 'document' groups
existing within the document.\n\nExample:\n- \\begin{document} ... \
\\end{document} ... \\begin{document} ... \\end{document} which can be fixed by
either combining the groups, or removing one of them, both ending in the form \
\\begin{document} ... \\end{document}", newStructureError)

USEPACKAGE_OUTSIDE_PREAMBLE = CustomErrorWrapper("\\usepackage should
only exist in the preamble.", "\\usepackage should only exist in the preamble.
\n\nExample:\n- \\documentclass{...} \\begin{document} ... \\usepackage{...} ...
\\end{document} can be fixed by moving the package to the preamble: \
\\documentclass{...} ... \\usepackage{...} ... \\begin{document} ... \
\\end{document}", newStructureError)

USEPACKAGE_LACKS_CLASS_ARGUMENT = CustomErrorWrapper("\\usepackage should
have a required argument. For example, '\\usepackage{graphicx}' is a valid
example.", "\\usepackage should have a required argument. A required argument is
an argument surrounded by curly braces '{}'. \n\nExample:\n- \\usepackage by
itself can cause the error, with \\usepackage{graphicx} being a valid package,
as 'graphicx' is a valid package name.", newStructureError)

/**
 * Accessibility related error functions
 */
// tagging accessibility errors
DOCUMENT_METADATA_MISSING = CustomErrorWrapper("Missing \
\\DocumentMetadata{tagging=on} line before \\documentclass.", "The document
metadata is not provided. This is necessary to ensure appropriate tagging of the
document takes place. A recommended document metadata command is as follows:
\n\n\\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)

DOCUMENT_METADATA_APPEARED_LATE = CustomErrorWrapper("\
\\DocumentMetadata should not appear after \\documentclass.", "The document
metadata must only exist before the document class.\n\nExample:\n- \
\\documentclass{...} ... \\DocumentMetadata{...} is invalid due to its ordering,
yet \\DocumentMetadata{...} ... \\documentclass{...} is valid",
newAccessibilityError)

DOCUMENT_METADATA_REPEATED = CustomErrorWrapper("There should
only be one \\DocumentMetadata in a single LaTeX file.", "You should only
contain 1 document metadata in your document.\n\nExample:\n- \
\\DocumentMetadata{...} ... \\DocumentMetadata{...} is invalid as there is more

```

```

than 1 document metadata, yet we can remove or combine them to create an
effective \\DocumentMetadata{...}", newAccessibilityError)
    METADATA_SEVERAL_ARGUMENTS = CustomErrorWrapper("\
\\DocumentMetadata should only have one required argument. For example, '\\
\\DocumentMetadata{tagging=on}'.", "The document metadata cannot contain more
than 1 argument, and its only argument should be a required one (surrounded by
curly braces '{}'). A recommended document metadata command is as follows:\n\n\
\\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
    METADATA_OTHER_ARGUMENTS_UNSUPPORTED = CustomErrorWrapper("\
\\DocumentMetadata should not have other argument types, such as those surrounded
by square brackets '[]'.", "The document metadata should not contain other
arguments types, such as optional arguments surrounded by square brackets. It
should contain only one argument which is a required one (surrounded by curly
braces '{}'). A recommended document metadata command is as follows:\n\n\
\\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
    METADATA_LACKS_CLASS_ARGUMENT = CustomErrorWrapper("\
\\DocumentMetadata should have one required argument.", "The document metadata
lacks a required argument. There should be exactly one required argument
(surrounded by curly braces '{}') passed to \\DocumentMetadata. A recommended
document metadata command is as follows:\n\n\\DocumentMetadata{\n\ttagging = on,
\n\ttagging-setup = {activate/all},\n\tlang=en-GB,\n\tpdfstandard =
ua-2,\n\tpdfversion = 2.0\n}", newAccessibilityError)
    METADATA_LACKS_ENABLED_TAGGING_ARGUMENT = CustomErrorWrapper("\
\\DocumentMetadata should have 'tagging=on' within its required argument.", "The
document metadata should contain the pair with key 'tagging' and value 'on'. A
recommended document metadata command is as follows:\n\n\
\\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
    METADATA_LACKS_TAGGING_SETUP_ARGUMENT = CustomErrorWrapper("\
\\DocumentMetadata should have 'tagging-setup={...}' within its required
argument. It is recommended to use 'tagging-setup={activate/all}'.", "The
document metadata should contain the pair with key 'tagging-setup' and a value,
surrounded by curly braces '{}', obtained from
https://ctan.math.washington.edu/tex-archive/macros/latex/contrib/tagpdf/
tagpdf.pdf. A recommended document metadata command is as follows:\n\n\
\\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
    METADATA_LACKS_PDF_STANDARD_ARGUMENT = CustomErrorWrapper("\
\\DocumentMetadata should have 'pdfstandard=...' within its required argument. It
is recommended to use 'pdfstandard=ua-2'.", "A PDF standard should be specified.
This utilises the key 'pdfstandard' and a value based on available standards. A
recommended document metadata command is as follows:\n\n\
\\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
    METADATA_LACKS_LANGUAGE_ARGUMENT = CustomErrorWrapper("\
\\DocumentMetadata should have 'lang=...' within its required argument. For
example, 'lang=en-GB' is a valid option.", "The language of the document should
be provided using the key 'lang' and an appropriate value from
https://developer.mozilla.org/en-US/docs/Glossary/BCP\_47\_language\_tag. A
recommended document metadata command is as follows:\n\n\
\\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
    METADATA_LACKS_PDF_VERSION = CustomErrorWrapper("\
\\DocumentMetadata should have 'pdfversion=...' within its required argument. It
is recommended to use 'pdfversion=2.0' for compatibility with
pdfstandard=ua-2.", "A suitable PDF version should be provided, through the key

```

'pdfversion' and value chosen based on the PDF standard the document utilises. A recommended document metadata command is as follows:\n\n\ DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}", newAccessibilityError)

```
// image-related alt text accessibility errors
GRAPHICS_MISSING_ARGUMENTS = CustomErrorWrapper("\
\includegraphics should at least 1 argument.", "\includegraphics requires at
least one argument for the file location of the graphic.\n\nExample:\n- \
\includegraphics is invalid by itself, whereas \includegraphics{image.png} is a
valid example", newAccessibilityError)
GRAPHICS_TOO_MANY_ARGUMENTS = CustomErrorWrapper("\
\includegraphics should have at most 2 arguments.", "\includegraphics currently
has more than 2 arguments, when the most arguments that can be supported is 2
(with an optional and required argument each).\n\nExample:\n- \
\includegraphics[alt={Some alt text}]{image.png} is an example of using 2
arguments to specify 'Some alt text' alt text, and 'image.png' file location",
newAccessibilityError)
GRAPHICS_FIRST_ARGUMENT_NOT_OPTIONAL = CustomErrorWrapper("\
\includegraphics should have its first of both arguments being optional. For
example, '\includegraphics[...]{...}' is a valid format.", "The first argument
of \includegraphics, in the context of having 2 arguments, should be an
optional argument (surrounded by square brackets '[']').\n\nExample:\n- \
\includegraphics{alt={Some alt text}}{image.png} is invalid as its first
argument should be optional, creating \includegraphics[alt={Some alt text}]
{image.png}", newAccessibilityError)
GRAPHICS_SECOND_ARGUMENT_NOT_REQUIRED = CustomErrorWrapper("\
\includegraphics should have its second of both arguments being required. For
example, '\includegraphics[...]{...}' is a valid format.", "The second argument
of \includegraphics, in the context of having 2 arguments, should be a required
argument (surrounded by curly braces '{}').\n\nExample:\n- \
\includegraphics[alt={Some alt text}]{image.png} is invalid as its second
argument should be required, creating \includegraphics[alt={Some alt text}]
{image.png}", newAccessibilityError)
GRAPHICS_OUTSIDE_DOCUMENT_CONTENT = CustomErrorWrapper("\
\includegraphics should only appear within the document content.", "\
\includegraphics cannot be contained outwith the document content, i.e. the
'document' group.\n\nExample:\n- \includegraphics[...]{...} ... \
\begin{document} ... \end{document} is invalid, whereas \begin{document} ... \
\includegraphics[...]{...} ... \end{document} is valid", newAccessibilityError)
GRAPHICS_LACKS_ALT_TEXT = CustomErrorWrapper("\
\includegraphics should contain an optional argument containing alternative text
('alt text'), actual text, or artifact. For example, '\includegraphics[alt={An
image of an apple}]{apple.png} is an accessible graphic.", "\includegraphics
should contain an optional argument containing alternative text ('alt text'),
actual text, or artifact.\n\nAlternative text is text to be used by screen
readers, or if the relevant graphic does not load.\nActual text is text that
replaces the image from a screen reader's perspective. It is read as ordinary
text.\nArtifact is used to ignore the image when read by a screen reader. This
is used in the context of decorative images.\n\nSome examples include:\n- \
\includegraphics[alt={An image of an apple}]{apple.png}\n- \
\includegraphics[actualtext={Orange}]{orange.png}\n- \includegraphics[artifact]
{luigi.png}", newAccessibilityError)
GRAPHICS_LACKS_SOURCE = CustomErrorWrapper("\
\includegraphics should contain a required argument for its source. For example,
'\includegraphics[alt={Some alt text}]{image.png}' is a valid graphic.", "\
\includegraphics contains exactly one optional argument, when an additional
argument for the source was expected.\n\nExample:\n- \includegraphics[alt={Some
alt text}] is invalid, but adding a source to create \includegraphics[alt={Some
alt text}]{image.png} makes it valid", newAccessibilityError)
MISSING_GRAPHICX_PACKAGE = CustomErrorWrapper("\
\includegraphics requires the 'graphicx' package. This can be included by
inserting '\usepackage{graphicx}' into the preamble.", "The 'graphicx' package
```

is required to use `\includegraphics`. This should be included in the preamble.
`\n\nExample:\n- \documentclass{...} ... \usepackage{graphicx} ... \begin{document} ... \includegraphics[alt={Some alt text}]{image.png} ... \end{document}`", newAccessibilityError)

// table-related accessibility errors

TABLE_LACKS_REQUIRED_ARGUMENT =

CustomErrorWrapper("Tabular groups should have an additional required argument.", "Tabular groups should contain an additional required argument indicating the structure of its columns. This occurs immediately after `\begin{tabular}`.
`\n\nExample:\n\\begin{tabular}{ccc}\n\tHeading 1 & Heading 2 & Heading 3 \\\\\n\\hline\n\t1 & 2 & 3 \\\\\n\t4 & 5 & 6\\\\\n\\end{tabular}`", newAccessibilityError)

TABLE_CANNOT_PARSE_COLUMNS =

CustomErrorWrapper("Could not parse how many columns the table has.", "unused.", newAccessibilityError)

TABLE_MISSING_TAG_PDF_SETUP =

CustomErrorWrapper("Tabular groups should have `\tagpdfsetup{...}` the line before the group. It should contain 'table/tagging=presentation' or 'table/header-rows={N}' where N is the row the heading occurs.", "Tabular groups should have `\tagpdfsetup{...}` the line before the group. It should contain 'table/tagging=presentation' or 'table/header-rows={N}' where N is the row the heading occurs. 'table/tagging=presentation' should be used when you do not want the screen reader to read the relevant heading before each table cell, whereas 'table/header-rows={N}' should be used if you do want that effect. N should be 1-indexed, such that the first row is N = 1.
`\n\nExample 1:\n\\tagpdfsetup{table/header-rows={1}}\n\\begin{tabular}{ccc}\n\tHeading 1 & Heading 2 & Heading 3 \\\\\n\\hline\n\t1 & 2 & 3 \\\\\n\t4 & 5 & 6\\\\\n\\end{tabular}\n\nExample 2:\n\\tagpdfsetup{table/tagging=presentation}\n\\begin{tabular}{ccc}\n\tHeading 1 & 1 & 2 \\\\\n\tHeading 2 & 3 & 4\\\\\n\\end{tabular}`", newAccessibilityError)

TAG_PDF_SETUP_REQUIRED_ARGUMENT = CustomErrorWrapper("\

`\tagpdfsetup` should have exactly one required argument.", "\tagpdfsetup should have exactly one required argument. A required argument is an argument surrounded by curly braces '{}'. It should contain 'table/tagging=presentation' or 'table/header-rows={N}' where N is the row the heading occurs. 'table/tagging=presentation' should be used when you do not want the screen reader to read the relevant heading before each table cell, whereas 'table/header-rows={N}' should be used if you do want that effect. N should be 1-indexed, such that the first row is N = 1.
`\n\nExamples:\n- \\\tagpdfsetup{table/tagging=presentation}\n- \\\tagpdfsetup{table/header-rows={1}}` when the table's headings are in the first row.", newAccessibilityError)

TAG_PDF_SETUP_NON_REQUIRED_ARGUMENT = CustomErrorWrapper("\

`\tagpdfsetup` should have an argument surrounded by curly braces '{}'.", "\tagpdfsetup cannot have optional arguments, instead it should have exactly one required argument. A required argument is an argument surrounded by curly braces '{}'. It should contain 'table/tagging=presentation' or 'table/header-rows={N}' where N is the row the heading occurs. 'table/tagging=presentation' should be used when you do not want the screen reader to read the relevant heading before each table cell, whereas 'table/header-rows={N}' should be used if you do want that effect. N should be 1-indexed, such that the first row is N = 1.
`\n\nExamples:\n- \\\tagpdfsetup{table/tagging=presentation}\n- \\\tagpdfsetup{table/header-rows={1}}` when the table's headings are in the first row.", newAccessibilityError)

TAG_PDF_SETUP_HEADER_ROWS_INVALID_VALUE = CustomErrorWrapper("\

`\tagpdfsetup{table/header-rows=...}` should have an integer value surrounded by curly braces as the inner argument. For example, `\tagpdfsetup{table/header-rows={1}}` indicates the first row is the heading row.", "\tagpdfsetup{table/header-rows=...} should have a value of form '{N}' where N is the row the headings occur.
`\n\nExample:\n- \\\tagpdfsetup{table/header-rows={1}}` where the headings occur in the first row", newAccessibilityError)

TAG_PDF_SETUP_LACKS_HEADER_ROWS_OR_PRESENTATION = CustomErrorWrapper("\

`\tagpdfsetup` should have an argument with 'table/header-rows={N}' with N being the row the heading occurs, or 'table/tagging=presentation' to indicate the

```

screen reader is not to read the table.", "\\tagpdfsetup lacks an appropriate
setting within its required argument. It should contain
'table/tagging=presentation' or 'table/header-rows={N}' where N is the row the
heading occurs. 'table/tagging=presentation' should be used when you do not want
the screen reader to read the relevant heading before each table cell, whereas
'table/header-rows={N}' should be used if you do want that effect. N should be
1-indexed, such that the first row is N = 1.\\n\\nExamples:\\n- \\
\\tagpdfsetup{table/tagging=presentation}\\n- \\tagpdfsetup{table/header-rows={1}}
when the table's headings are in the first row.", newAccessibilityError)
    TAG_PDF_SETUP_TABLE_TAGGING_INVALID_VALUE = CustomErrorWrapper("\\
\\tagpdfsetup{table/tagging=...} should have the inner value 'presentation'
creating '\\tagpdfsetup{table/tagging=presentation}'.",
"\\tagpdfsetup{table/tagging=...} contains an invalid value. The expected value
was 'presentation' such that \\tagpdfsetup{table/tagging=presentation} would be
the result.", newAccessibilityError)

/**
 * Server-created errors. These can be caused by server-related processes
failing
 */
    SERVER_RESPONSIBLE_CANNOT_FIND_WORKING_DIRECTORY =
CustomErrorWrapper("Something went wrong on the server's end. The working
directory could not be retrieved. Please report this to the server owner.",
"Something went wrong on the server's end. The working directory could not be
retrieved. Please report this to the server owner.", newServerError)
    SERVER_RESPONSIBLE_WRITE_FILE_ERROR =
CustomErrorWrapper("Something went wrong on the server's end. Could not write to
the file. Please report this to the server owner.", "Something went wrong on the
server's end. Could not write to the file. Please report this to the server
owner.", newServerError)
    SERVER_RESPONSIBLE_READ_FILE_ERROR =
CustomErrorWrapper("Something went wrong on the server's end. The file location
for the LaTeX file could not be read. Please report this to the server owner.",
"Something went wrong on the server's end. The file location for the LaTeX file
could not be read. Please report this to the server owner.", newServerError)
    SERVER_RESPONSIBLE_UNIDENTIFIED_ERROR =
CustomErrorWrapper("Something went wrong on the server's end. There is an
unidentified error. Please report this to the server owner.", "Something went
wrong on the server's end. There is an unidentified error. Please report this to
the server owner.", newServerError)
    SERVER_RESPONSIBLE_INVALID_STRING_ERROR =
CustomErrorWrapper("Something went wrong on the server's end. Could not parse
string. Please report this to the server owner.", "Something went wrong on the
server's end. Could not parse string. Please report this to the server owner.",
newServerError)
    SERVER_RESPONSIBLE_INVALID_REGEX_ERROR =
CustomErrorWrapper("Something went wrong on the server's end. Could not parse
regular expression. Please report this to the server owner.", "Something went
wrong on the server's end. Could not parse regular expression. Please report
this to the server owner.", newServerError)
)

```